

Cours complet — Programmation Fonctionnelle en Dart

Presentation du cours :

bienvenu dans ce cours de la programmation fonctionnelle en dart Vous allez découvrir une manière révolutionnaire de coder avec Dart et Flutter, qui rend votre code clair, sûr, réutilisable et élégant. Ce cours vous guide pas à pas pour maîtriser le paradigme fonctionnel, un atout indispensable pour construire des applications robustes, maintenables et modernes.

Que vous soyez débutant ou développeur confirmé, vous apprendrez à écrire un code prévisible, puissant et facile à tester, et à transformer vos applications Dart et Flutter en projets professionnels et fiables.

Ce que vous allez apprendre

1. Fonctions de première classe

Stocker, passer et retourner des fonctions.

Fonctions d'ordre supérieur

Passer une fonction en paramètre ou retourner une fonction.

Fonctions anonymes et lambdas

Simplifier le code pour des opérations rapides et expressives.

Fonctions pures vs impures

Apprendre à isoler la logique pour un code testable.

Immutabilité

Créer des données fixes et sécurisées.

Méthodes fonctionnelles sur les collections

Utiliser map, where, fold pour transformer vos listes de manière élégante.

Currying et application partielle

Créer des fonctions spécialisées pour simplifier les appels.

Introduction au package dartz

Le package dartz permet d'appliquer la PF de manière pratique en Dart et Flutter :

Projet pratique Flutter

- Créer un service API utilisant Either pour gérer les erreurs.

- Afficher les données dans une UI Flutter de manière fonctionnelle.
- Ce projet montre comment **appliquer la PF dans le monde réel**, pour des applications fiables et maintenables.

Introduction

Dart est avant tout un langage **orienté objet**, mais il supporte aussi les **principes de la programmation fonctionnelle (PF)**.

La programmation fonctionnelle considère le calcul comme l'**évaluation de fonctions pures**, en évitant toute **modification d'état partagée**.

La **programmation fonctionnelle (FP)** repose sur un principe simple : écrire des fonctions pures, sans modifier l'état global, afin d'obtenir un code **prévisible, testable et robuste**.

Pourquoi utiliser la programmation fonctionnelle en Dart ?

Lisibilité améliorée : le code est plus clair et prévisible. car chaque fonction est indépendante.

Tests plus faciles : les fonctions pures n'ont pas d'effets de bord. donne toujours le même résultat pour les mêmes entrées

Moins d'erreurs en multithreading : pas d'état partagé mutable. pas de conflits entre threads.

Moins de complexité : le code est plus modulaire et réutilisable.

?? Partie 1 — Fonctions de première classe

En Dart, les fonctions sont des citoyens de première classe :

- Elles peuvent être stockées dans des variables.
- Passées en paramètres à d'autres fonctions.
- Ou retournées comme valeurs.

?? Exemple simple — Affecter une fonction à une variable

```
// ?? Définition d'une fonction simple qui prend un nom en paramètre  
// et affiche un message de salutation
```

```

void direBonjour(String nom) {
    print("Bonjour, $nom !");
}

void main() {
    // ❖❖ Stockage de la fonction dans une variable
    // Ici, 'salutation' contient la référence à la fonction 'direBonjour',
    // pas le résultat de son exécution.
    var salutation = direBonjour;

    // ❖❖ Exécution de la fonction via la variable
    // On passe "Ciko" comme argument, la fonction affiche le message
    salutation("Ciko"); // Affiche : Bonjour, Ciko !
}

```

Fonctions de première classe

- En Dart, les fonctions peuvent être **assignées à des variables**, passées en paramètre ou retournées par d'autres fonctions.
- Ici, salutation est une variable qui **contient la fonction** direBonjour.

Référence vs exécution

- var salutation = direBonjour; → stocke la référence, **sans exécuter** la fonction.
- salutation("Ciko"); → exécute la fonction avec l'argument "Ciko".

Avantage

- Permet de **passer dynamiquement des comportements** et d'écrire du code plus flexible et réutilisable.

❖❖ Ici, salutation contient la référence de la fonction direBonjour, pas son résultat.

❖❖ Exemple avancé — Fonctions dynamiques

```

// ❖❖ Définition de deux fonctions simples qui prennent un message en
paramètre
// - afficher : affiche simplement le message
void afficher(String message) => print(message);

// - avertir : affiche le message avec un symbole d'avertissement
void avertir(String message) => print("    $message");

void main() {

```

```
// ❖❖ Variable booléenne qui indique si l'action est urgente
bool urgent = true;

// ❖❖ Choix dynamique de la fonction à exécuter selon la condition
// Si urgent est vrai, on choisit 'avertir', sinon 'afficher'
var action = urgent ? avertir : afficher;

// ❖❖ Exécution de la fonction sélectionnée
// Ici, comme urgent = true, la fonction 'avertir' est appelée
action("Attention à bien tester ton code !");

// Affiche :      Attention à bien tester ton code !
}
```

Explications pédagogiques

1. Fonctions de première classe

- En Dart, les fonctions peuvent être **stockées dans des variables**, ce qui permet de les appeler plus tard.
- Ici, action est une variable qui contient soit avertir, soit afficher.

2. Choix dynamique

- L'opérateur ternaire ? : permet de **sélectionner la fonction à exécuter** en fonction d'une condition.

3. Avantage

- Permet d'écrire du code **plus flexible et réutilisable**, car le comportement peut changer au moment de l'exécution sans modifier la logique de la fonction principale.

❖❖ Bonnes pratiques :

- Ne pas exécuter la fonction avec () lors de l'affectation.
- Toujours garder un nom explicite pour la fonction.

❖❖ Partie 2 — Fonctions d'ordre supérieur (Higher-Order Functions)

Une **fonction d'ordre supérieur** est une fonction qui :

- prend une autre fonction en paramètre,
- ou retourne une fonction.

❖❖ Exemple simple — Fonction retournant une autre fonction

```
// ❖❖ Définition d'une fonction qui renvoie une autre fonction
```

```
// La fonction multiplierPar prend un entier 'facteur' en paramètre
// et retourne une fonction anonyme qui multiplie son argument par ce facteur
Function multiplierPar(int facteur) {
    return (int nombre) => nombre * facteur;
}

void main() {
    // ❖❖ Création d'une fonction spécialisée 'doubler'
    // multiplierPar(2) renvoie une fonction qui multiplie par 2
    var doubler = multiplierPar(2);

    // ❖❖ Appel de la fonction retournée
    // Ici, doubler(5) calcule 5 * 2 = 10
    print(doubler(5)); // Affiche : 10
}
❖❖ Ici, multiplierPar(2) crée une fonction spécialisée pour multiplier par 2.
```

Explications pédagogiques

1. **Fonctions d'ordre supérieur**
 - multiplierPar est une fonction **d'ordre supérieur** car elle **retourne une autre fonction**.
 - Cela permet de créer des fonctions spécialisées à partir d'une fonction générale.
2. **Fonctions anonymes (lambdas)**
 - La fonction (int nombre) => nombre * facteur n'a pas de nom, c'est une **fonction anonyme**.
 - Elle capture la variable facteur depuis la fonction parente (closure).
3. **Avantage pratique**
 - Permet de **réutiliser le même code** pour créer plusieurs fonctions spécialisées, par exemple doubler, tripler, etc., sans répéter la logique.

❖❖ Exemple avancé — Transformation de liste avec fonction passée en argument

```
// ❖❖ Définition d'une fonction de haut niveau (Higher-Order Function)
// Cette fonction prend en paramètre :
// - une liste de nombres entiers (List<int> nombres)
// - une fonction (operation) qui prend un int et retourne un int (int
Function(int))
void appliquerSurListe(List<int> nombres, int Function(int) operation) {
```

```

// ❖❖ On parcourt chaque élément de la liste
for (var n in nombres) {
  // ❖❖ On applique la fonction "operation" sur chaque nombre
  // et on affiche le résultat
  print(operation(n));
}
}

// ❖❖ Définition d'une fonction simple qui double un nombre
int doubler(int n) => n * 2;

// ❖❖ Définition d'une autre fonction simple qui ajoute 3 à un nombre
int ajouterTrois(int n) => n + 3;

void main() {
  // ❖❖ On appelle "appliquerSurListe" en passant :
  // - la liste [1, 2, 3]
  // - la fonction "doubler"
  appliquerSurListe([1, 2, 3], doubler);
  // ❖❖ Résultat affiché :
  // 2
  // 4
  // 6

  // ❖❖ Cette fois, on applique la fonction "ajouterTrois"
  appliquerSurListe([1, 2, 3], ajouterTrois);
  // ❖❖ Résultat affiché :
  // 4
  // 5
  // 6
}

```

Explication pédagogique

1. **Fonction de haut niveau (Higher-Order Function)**
 Une fonction est dite “de haut niveau” lorsqu’elle prend une autre fonction en paramètre **ou** qu’elle retourne une fonction.
 → Ici, appliquerSurListe prend une fonction operation comme paramètre.
2. **Fonctions comme valeurs**
 En Dart (comme dans la programmation fonctionnelle), les fonctions peuvent être **stockées dans des variables, passées en paramètres, ou retournées** comme des objets.
 → Exemple : doubler et ajouterTrois sont passées comme arguments.
3. **Réutilisabilité et clarté**
 Grâce à ce modèle, on peut créer **une seule fonction générique**

(appliquerSurListe) et l'utiliser avec **n'importe quelle transformation** (doubler, ajouterTrois, diviserParDeux, etc.).

Bonne pratique : toujours typer les fonctions en paramètre (int Function(int)).

❖❖ Partie 3 — Fonctions anonymes (Lambda ou Arrow Functions)

Les **fonctions anonymes** n'ont pas de nom.

Elles sont idéales pour des opérations courtes et expressives.

❖❖ Exemple simple

```
void main() {  
    // ❖❖❖ Déclaration d'une fonction anonyme (lambda ou arrow function)  
    // Ici, "carre" est une variable qui contient une fonction.  
    // La fonction prend un entier "x" et retourne x * x.  
    var carre = (int x) => x * x;  
  
    // ❖❖❖ Appel de la fonction stockée dans la variable "carre"  
    // On passe 4 comme argument, donc le résultat sera 4 * 4 = 16  
    print(carre(4)); // Affiche : 16  
}
```

Explication pédagogique

1. Fonction anonyme (lambda)

- Une fonction anonyme est **une fonction sans nom**.
- On peut l'assigner directement à une variable.
- Syntaxe courte : (paramètres) => expression;

2. Fonctions comme valeurs

- Ici, carre n'est **pas le résultat** de la fonction, mais **la fonction elle-même**.
- On peut donc la réutiliser plusieurs fois avec différents arguments :

- print(carre(5)); // 25
- print(carre(10)); // 100
-

3. Avantages

- Code concis et lisible.
- Idéal pour des opérations simples et ponctuelles.
- Peut être passée en argument à d'autres fonctions pour la programmation fonctionnelle.

💡💡 Exemple avancé — Utilisation avec forEach

```
void main() {  
  // 💡💡 Création d'une liste de noms  
  var noms = ["Ciko", "Awa", "John"];  
  
  // 💡💡 Utilisation de forEach avec une fonction anonyme  
  // Pour chaque élément de la liste "noms", on exécute la fonction passée  
  // Ici, on affiche "Bonjour <nom> 💡💡"  
  noms.forEach((nom) {  
    print("Bonjour $nom 💡💡");  
  });  
  
  // 💡💡 Version raccourcie avec une arrow function (fonction fléchée)  
  // Syntaxe plus concise pour des actions simples  
  noms.forEach((nom) => print("Salut $nom !"));  
}
```

Explication pédagogique

1. forEach

- C'est une méthode des collections en Dart (liste, set, etc.)
- Elle permet d'exécuter une fonction pour **chaque élément** de la collection.

2. Fonctions anonymes vs arrow functions

- (nom) { print(...); } → fonction anonyme classique
- (nom) => print(...); → version fléchée, plus concise pour des expressions simples

3. Avantages

- Code **lisible et fonctionnel**, pas besoin de boucle for.
- Permet de combiner la **programmation fonctionnelle** avec les collections.

💡💡 *Les lambdas rendent le code plus concis, mais évite d'y mettre trop de logique.*

❖❖ Partie 4 — Fonctions pures

Une **fonction pure** :

- renvoie toujours le même résultat pour les mêmes entrées,
- n'a **aucun effet de bord** (elle ne modifie pas de variable externe).

Exemple de fonction pure

```
// ❖❖ Déclaration d'une fonction pure nommée "addition"
// Elle prend deux entiers en entrée (a et b) et retourne leur somme
int addition(int a, int b) {
    return a + b; // Renvoie la somme de a et b
}

void main() {
    // ❖❖ Appel de la fonction "addition" avec les valeurs 3 et 5
    // Comme c'est une fonction pure, le résultat sera toujours le même pour ces
    mêmes entrées
    print(addition(3, 5)); // Affiche : 8
}
```

Explications pédagogiques

1. **Fonction pure**
 - Toujours le même résultat pour les mêmes entrées.
 - Ne modifie **aucune variable externe** et n'a **aucun effet de bord**.
2. **return**
 - Permet de renvoyer le résultat calculé par la fonction.
3. **main()**
 - Fonction principale qui s'exécute au démarrage du programme.
 - Ici, elle sert à tester la fonction addition.
4. **Avantage**
 - La pureté de la fonction rend le code **prévisible, testable et facile à déboguer**.

Exemple de fonction impure

```
// ❖❖ Déclaration d'une variable globale "compteur"
// Cette variable est accessible et modifiable depuis n'importe quelle fonction
int compteur = 0;

// ❖❖ Déclaration d'une fonction "incrémenter" qui modifie l'état externe
```

```

void incrementer() {
    compteur++; // Effet de bord : la fonction modifie la variable globale
}

void main() {
    // ❖❖ Appel de la fonction "incrementer"
    incrementer(); // Le compteur passe de 0 à 1
    print(compteur); // Affiche : 1
}

```

Explications pédagogiques

1. Fonction impure

- Modifie une **variable externe** (compteur) au lieu de ne dépendre que de ses paramètres.
- Le résultat dépend de l'état global, donc **imprévisible** si plusieurs fonctions modifient la même variable.

2. Effets de bord (Side Effects)

- Toute modification d'une variable globale ou statique, ou toute opération externe (écriture sur disque, requête réseau, etc.), constitue un effet de bord.

3. Inconvénient

- Rend le code **difficile à tester et à maintenir** car l'état global peut changer n'importe quand.

❖❖ *Les fonctions impures rendent le code difficile à tester et à maintenir.*

❖❖ Partie 5 — Immutabilité

L'**immutabilité** consiste à **ne jamais modifier directement** une donnée existante.

❖❖ Exemple simple

```

void main() {
    // ❖❖ Déclaration d'une variable finale "nom"
    // Une variable final ne peut être assignée qu'une seule fois
    final nom = "Ciko";

    // nom = "Chris";
    // Cette ligne provoquerait une erreur car "nom" est final
    // On ne peut pas réassigner une valeur à une variable finale
}

```

```
// ❖❖ Déclaration d'une constante "pi"
// Une constante const est immuable et connue à la compilation
const pi = 3.14;

// ❖❖ Affichage des valeurs
print(nom); // Affiche : Ciko
print(pi); // Affiche : 3.14
}
```

Explications pédagogiques

1. **final**
 - Une variable final **ne peut être assignée qu'une seule fois**.
 - Sa valeur est définie à l'**exécution**, mais **ne peut pas changer après**.
2. **const**
 - Une constante const **est immuable et connue au moment de la compilation**.
 - Elle est souvent utilisée pour des valeurs fixes comme pi, couleurs, ou paramètres globaux.
3. **Avantages de l'immutabilité**
 - Évite les erreurs liées à la modification involontaire de données.
 - Facilite la **programmation fonctionnelle**, car les fonctions peuvent rester **pures**.

❖❖ Exemple avancé — Liste immuable

```
void main() {
// ❖❖ Création d'une liste immuable
// List.unmodifiable crée une liste en lecture seule, impossible de la modifier
final nombres = List.unmodifiable([1, 2, 3]);

// nombres.add(4);
// Cette ligne provoquerait une erreur car la liste est immuable (lecture seule)

// ❖❖ Affichage de la liste
print(nombres); // Affiche : [1, 2, 3]
}
```

Explications pédagogiques

1. **List.unmodifiable**

- Crée une **copie immuable** d'une liste.
 - Toute tentative de modification (add, remove, clear, etc.) déclenche une erreur.
2. **final + List.unmodifiable**
 - final empêche la réassignation de la variable nombres.
 - List.unmodifiable empêche la modification de son contenu.
 3. **Avantages**
 - Sécurité : pas de modification accidentelle de données partagées.
 - Programmation fonctionnelle : favorise des fonctions pures et un code prévisible.

❖❖ *Créer une nouvelle liste au lieu de modifier l'existante :*

```
var nouvelleListe = [...nombres, 4];
print(nouvelleListe); // [1, 2, 3, 4]
```

❖❖ Partie 6 — Fonctions sur les collections

Dart offre plusieurs méthodes fonctionnelles sur les listes :

Méthode	Description	Exemple
map()	transforme les éléments	doubler chaque nombre
where()	filtre selon une condition	garder les pairs
reduce()	combine les valeurs	calculer la somme
fold()	combine avec une valeur initiale	somme avec 0

❖❖ Exemple complet

```
void main() {
  // ❖❖ Liste initiale
  var nombres = [1, 2, 3, 4, 5];

  // ❖❖ Transformation fonctionnelle avec chaînage de méthodes
  var resultat = nombres
    .map((n) => n * 2) // ❖❖ Double chaque élément : [2, 4, 6, 8, 10]
    .where((n) => n > 5) // ❖❖ Filtre les éléments > 5 : [6, 8, 10]
    .map((n) => n + 1) // ❖❖ Ajoute 1 à chaque élément : [7, 9, 11]
    .toList(); // ❖❖ Convertit le résultat en liste

  // ❖❖ Affichage final
  print(resultat); // Affiche : [7, 9, 11]
}
```

Explications pédagogiques

1. map()

- Transforme chaque élément de la liste selon la fonction passée.
 - Exemple : $n * 2$ double chaque nombre.
2. **where()**
 - Filtre les éléments selon la condition donnée.
 - Ici : on ne garde que les nombres strictement supérieurs à 5.
 3. **Chaînage des méthodes**
 - Les méthodes sont appelées **les unes après les autres**, créant un pipeline de transformations sans modifier la liste originale.
 - Cela respecte le principe de **programmation fonctionnelle** : code clair, concis et immuable.
 4. **toList()**
 - Convertit l'itérateur résultant en liste concrète pour pouvoir l'utiliser facilement.

Code concis, lisible, sans boucle for.

Partie 7 — Currying (Application partielle)

Le **currying** consiste à décomposer une fonction à plusieurs paramètres en plusieurs fonctions à un seul paramètre.

💡💡 Exemple simple

```
// 💡💡 Fonction 'addition' qui prend un entier 'a' et retourne une autre fonction
// Cette fonction retournée prend un entier 'b' et renvoie la somme a + b
Function addition(int a) {
    return (int b) => a + b;
}
```

```
void main() {
    // 💡💡 On crée une fonction spécialisée qui ajoute toujours 5
    var ajouter5 = addition(5);
```

```
    // 💡💡 On utilise cette fonction pour ajouter 3 à 5
    print(ajouter5(3)); // Affiche : 8
}
```

💡💡 Ici, ajouter5 est une fonction spécialisée qui ajoute toujours 5.

Explications pédagogiques

1. Currying / Application partielle

- La fonction addition **ne fait pas directement l'addition**.
 - Elle **retourne une fonction** qui fera l'addition avec la valeur a déjà fixée.
2. **Création d'une fonction spécialisée**
 - ajouter5 est une fonction "préconfigurée" pour toujours ajouter 5.
 - Cela rend le code **modulaire et réutilisable**.
 3. **Exécution**
 - ajouter5(3) équivaut à $5 + 3$.
 - La valeur 5 est "capturée" par la fonction retournée (fermeture / closure).

Introduction à dartz

Le package **dartz** fournit des **outils de programmation fonctionnelle** pour Dart et Flutter.

Il inclut des structures comme :

- Option (valeur présente ou absente),
- Either (succès ou échec),
- Task et TaskEither (asynchronisme fonctionnel),
- Validation (chaînage de validations avec collecte d'erreurs).

Installation

Ajoute dartz dans ton fichier **pubspec.yaml** :

dependencies:

dartz: ^0.10.1

Puis lance la commande :

```
flutter pub get
```

❖❖ 1. Le type Option<T>

Le type Option représente **une valeur optionnelle** :

- Some(value) → la valeur est présente.
- None() → la valeur est absente.

Cela permet d'éviter les **null checks** répétitifs et les **NullPointerException**.

Exemple :

```
import 'package:dartz/dartz.dart';
```

```
void main() {
```

```
// ❖❖ Création d'une valeur présente (Some) contenant l'entier 10
```

```
Option<int> someValue = some(10);
```

```
// ❖❖❖ Création d'une valeur absente (None)
Option<int> noneValue = none();

// ❖❖❖ La méthode fold() permet de gérer les deux cas : Some ou None
someValue.fold(
    () => print('  Aucune valeur'),    // Action si la valeur est absente (None)
    (value) => print('  La valeur est $value'), // Action si la valeur est présente
(Some)
);

noneValue.fold(
    () => print('  Aucune valeur'),    // Ici None => affiche ce message
    (value) => print('  La valeur est $value'), // Ici Some => n'est pas exécuté
);
}
```

Explications pédagogiques

1. Option<T>

- Option est un conteneur pour une valeur **qui peut être présente (Some) ou absente (None)**.
- Cela remplace l'usage des null et des vérifications répétitives (if (value != null)).

2. fold()

- fold() prend deux fonctions :
 - Une pour le cas **None** (absence de valeur).
 - Une pour le cas **Some** (valeur présente).
- Cela permet de **traiter les deux cas de manière sûre et fonctionnelle**.

3. Avantages

- Évite les erreurs de type NullPointerException.
- Rend le code **plus lisible et prévisible**.
- Compatible avec la **programmation fonctionnelle**.

❖❖❖ Remarque :

Option remplace avantageusement le ? et les vérifications if (value != null).

2. Le type Either<L, R>

Either est un conteneur qui contient **soit une erreur (Left), soit une valeur valide (Right)**.

C'est très utilisé pour gérer les **erreurs sans try-catch**.

Exemple :

```
import 'package:dartz/dartz.dart';

void main() {
  // ❖❖ Création d'un succès (Right) contenant l'entier 10
  Either<String, int> success = right(10);

  // ❖❖ Création d'un échec (Left) contenant un message d'erreur
  Either<String, int> failure = left('Une erreur est survenue');

  // ❖❖ La méthode fold() permet de gérer les deux cas : Left (erreur) ou Right (succès)
  success.fold(
    (error) => print(' Erreur: $error'), // Action si c'est un Left (erreur)
    (value) => print(' Valeur: $value'), // Action si c'est un Right (succès)
  );

  failure.fold(
    (error) => print(' Erreur: $error'), // Ici Left => affiche le message d'erreur
    (value) => print(' Valeur: $value'), // Ici Right => n'est pas exécuté
  );
}
```

Explications pédagogiques

1. Either<L, R>

- Either est un conteneur qui peut contenir soit :
 - Left (généralement utilisé pour une erreur ou un échec)
 - Right (généralement utilisé pour un succès ou une valeur valide)
- Cela permet de **remplacer try/catch** dans la programmation fonctionnelle.

2. fold()

- fold() prend deux fonctions :
 - Une pour le cas Left (gérer l'erreur).
 - Une pour le cas Right (utiliser la valeur).
- Cela **force à traiter les deux cas**, rendant le code plus sûr et prévisible.

3. Avantages

- Évite les exceptions non gérées.
- Facilite la **gestion fonctionnelle des erreurs**.
- Rend le code **plus lisible et testable**.

❓❓ Either est parfait pour les **opérations qui peuvent échouer** (comme les appels API).

3. Task et TaskEither

Ces deux types permettent de gérer l'**asynchronisme de manière fonctionnelle**.

- Task encapsule une opération asynchrone.
- TaskEither encapsule une opération asynchrone **qui peut échouer**.

Exemple :

Dependancies:

fpdart: ^1.1.0

```
import 'package:fpdart/fpdart.dart';
```

```
void main() async {  
  // ❓❓ Task simple  
  // Task encapsule une opération asynchrone qui renvoie une valeur  
  Task<int> task = Task(() async => 42);  
  
  // run() exécute la tâche et retourne le résultat  
  int result = await task.run();  
  print('Résultat: $result'); // Résultat: 42  
  
  // ❓❓ TaskEither (succès ou erreur)  
  // TaskEither encapsule une opération asynchrone qui peut échouer  
  // Right = succès, Left = erreur  
  TaskEither<String, int> taskEither = TaskEither(() async => right(42));  
  
  // run() exécute la tâche asynchrone et renvoie un Either  
  Either<String, int> eitherResult = await taskEither.run();  
  
  // fold() permet de traiter les deux cas : Left (erreur) ou Right (valeur)
```

```

eitherResult.fold(
  (error) => print(' Erreur: $error'), // Cas d'erreur
  (value) => print(' Valeur: $value'), // Cas succès
);
}

```

Explications pédagogiques

1. **Task<T>**
 - Représente une opération **asynchrone pure** qui retourne une valeur de type T.
 - Avantage : permet de **composer des opérations asynchrones** de manière fonctionnelle.
2. **TaskEither<L, R>**
 - Comme Task, mais pour les opérations qui peuvent **échouer**.
 - Left contient l'erreur (L), Right contient le résultat (R).
 - Cela évite les exceptions et facilite le **traitement fonctionnel des erreurs asynchrones**.
3. **fold()**
 - Gère les deux scénarios (Left ou Right) de manière claire et **sans try/catch**.
 - Encourage la **programmation défensive et prévisible**.
4. **Avantages**
 - Gestion des erreurs et asynchronisme de façon **pure et fonctionnelle**.
 - Code plus **testable et maintenable**.
 - Permet de **chaîner plusieurs opérations asynchrones** proprement.

❖❖ 4. Validation

Validation permet de **combiner plusieurs vérifications** et de **collecter toutes les erreurs** (contrairement à Either qui s'arrête à la première erreur).

Exemple :

```

import 'package:fpdart/fpdart.dart';

void main() {
  // ❖❖ Validation d'une valeur positive

```

```
// Cette fonction vérifie si le nombre est > 0
// Si oui, on retourne Right(value) (succès)
// Sinon, Left(...) (erreur)
Either<String, int> validatePositive(int value) {
    return value > 0
        ? Either.right(value)          // Cas succès
        : Either.left(' La valeur doit être positive'); // Cas erreur
}
```

```
// ❖❖ Validation d'une valeur paire
// Cette fonction vérifie si le nombre est pair
// Right(value) si pair, Left(error) sinon
Either<String, int> validateEven(int value) {
    return value % 2 == 0
        ? Either.right(value)          // Cas succès
        : Either.left(' La valeur doit être paire'); // Cas erreur
}
```

```
// ❖❖ Combinaison de validations
// On enchaîne les deux validations avec flatMap :
// - Si la première validation échoue (Left), on s'arrête
// - Sinon, on continue avec la seconde validation
Either<String, int> validate(int value) {
    return validatePositive(value) // Vérifie si positif
        .flatMap((v) => validateEven(v)); // Puis vérifie si pair
}
```

```
// ❖❖ Tests des validations
print(validate(2)); // Right(2) : 2 est positif et pair
print(validate(-2)); // Left(...) : -2 n'est pas positif, donc erreur
print(validate(1)); // Left(...) : 1 est positif mais pas pair, donc erreur
}
```

Explications clés :

- `Either<L, R>` : type fonctionnel représentant **succès (Right)** ou **erreur (Left)**.
- `flatMap` : permet de **chaîner des opérations** qui retournent un `Either` et d'arrêter la chaîne en cas d'erreur.
- Cette approche rend le code **fonctionnel, prévisible et facile à tester**.

❖❖ 5. Exemple pratique avec Flutter

Étape 1 : Créer le projet

```
flutter create functional_flutter_app
cd functional_flutter_app
Ajoute dans pubspec.yaml :
```

```
dependencies:
  flutter:
    sdk: flutter
  dartz: ^0.10.1
  http: ^1.1.0
```

Étape 2 : Service API avec Either

api_service.dart

```
import 'package:dartz/dartz.dart';
import 'package:http/http.dart' as http;
import 'dart:convert';

// ❖❖❖ Classe de service pour récupérer des données depuis une API
class ApiService {
  // ❖❖❖ Méthode asynchrone fetchData
  // Elle renvoie un Either :
  // - Right(List<String>) si la récupération est réussie
  // - Left(String) si une erreur se produit
  Future<Either<String, List<String>>> fetchData() async {
    // ❖❖❖ Requête HTTP GET vers l'API
    final response = await
http.get(Uri.parse('https://jsonplaceholder.typicode.com/posts'));

    // ❖❖❖ Vérification du code HTTP
    if (response.statusCode == 200) {
      // Succès : on decode le JSON en liste dynamique
      List<dynamic> data = json.decode(response.body);

      // ❖❖❖ On extrait uniquement les titres et on les transforme en List<String>
      List<String> titles = data.map((item) => item['title'] as String).toList();

      // ❖❖❖ On retourne la liste dans un Either Right pour indiquer le succès
      return right(titles);
    } else {
```

```

//   Échec : on retourne un Either Left avec un message d'erreur
return left('   Échec de récupération des données');
}
}
}

```

Explications pédagogiques

1. Either<L, R>

- Conteneur fonctionnel pour gérer **succès ou échec**.
- Right(value) → succès
- Left(error) → échec

2. Avantages

- Évite les try/catch dispersés.
- Rend le code **plus lisible et fonctionnel**.
- On peut facilement **chaîner plusieurs opérations** tout en gérant les erreurs proprement.

3. Astuce

- Avec fold(), on peut traiter le succès et l'erreur de manière concise, ce qui est pratique pour l'intégration dans Flutter UI avec FutureBuilder.

Étape 3 : Intégration dans l'UI Flutter

main.dart

```

import 'package:flutter/material.dart';
import 'api_service.dart';
import 'package:dartz/dartz.dart';

```

```

//   Point d'entrée de l'application Flutter
void main() => runApp(MyApp());

```

```

//   Widget racine de l'application
class MyApp extends StatelessWidget {
  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Functional Programming in Flutter', // Titre de l'application
      home: HomePage(), // Écran principal
    );
  }
}

```

```

// ❖❖ Écran principal affichant les données de l'API
class HomePage extends StatelessWidget {
  // ❖❖ Instance du service API pour récupérer les données
  final ApiService apiService = ApiService();

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Functional Programming in Flutter')), // Barre
d'application
      body: FutureBuilder(
        // ❖❖ Future qui récupère les données depuis l'API
        future: apiService.fetchData(),
        builder: (context, snapshot) {
          // ❖❖ Affichage pendant le chargement
          if (snapshot.connectionState == ConnectionState.waiting) {
            return Center(child: CircularProgressIndicator());
          }
          // ❖❖ Affichage si une erreur a été rencontrée lors du Future
          else if (snapshot.hasError) {
            return Center(child: Text(' Erreur: ${snapshot.error}'));
          }
          // ❖❖ Affichage lorsque le Future a renvoyé des données
          else if (snapshot.hasData) {
            // ❖❖ snapshot.data contient un Either<String, List<String>>
            final Either<String, List<String>> result = snapshot.data as
Either<String, List<String>>;

            // ❖❖ fold() permet de gérer les deux cas : erreur (Left) ou succès
(Right)
            return result.fold(
              // ❖❖ Cas d'erreur : affichage du message
              (error) => Center(child: Text(error)),

              // ❖❖ Cas succès : affichage des titres dans une liste
              (data) => ListView.builder(
                itemCount: data.length,
                itemBuilder: (context, index) => ListTile(title: Text(data[index])),
              ),
            );
          }
          // ❖❖ Cas inattendu
          else {
            return Center(child: Text(' Erreur inconnue'));
          }
        }
      )
    );
  }
}

```

```

    },
  ),
);
}
}

```

Explications pédagogiques

1. **FutureBuilder**
 - Widget pratique pour construire l'UI en fonction d'un Future.
 - Permet de gérer l'état du chargement (waiting), des erreurs (hasError) et des données (hasData).
2. **Either.fold()**
 - Permet de **séparer clairement le traitement du succès et de l'erreur**.
 - Ici, on évite les try/catch et on garde le code fonctionnel.
3. **ListView.builder**
 - Affiche dynamiquement la liste des titres récupérés.
 - Optimisé pour les longues listes.
4. **Séparation logique**
 - ApiService s'occupe uniquement de la récupération des données.
 - HomePage gère l'affichage, respectant ainsi le **principe de séparation des responsabilités**.

Conclusion du cours de Programmation Fonctionnelle en Dart

Félicitations ! 💎💎 Vous avez maintenant exploré le **paradigme fonctionnel** et découvert comment il peut transformer votre façon de coder en Dart et Flutter.

Vous savez désormais :

- Utiliser des **fonctions de première classe**, des **fonctions pures** et des **fonctions anonymes**.
- Appliquer des concepts puissants comme **l'immuabilité**, les **méthodes fonctionnelles sur les collections**, le **currying** et l'**application partielle**.
- Gérer les erreurs et la logique de manière élégante grâce à des outils comme **Either** et **Option** avec le package fpdart.
- Concevoir des **applications robustes, testables et maintenables**, en appliquant la programmation fonctionnelle à des projets concrets Flutter.

La programmation fonctionnelle n'est pas juste une technique : c'est un **état d'esprit**, une façon de penser le code pour qu'il soit **prévisible, sûr et réutilisable**.

❖❖ Maintenant, c'est à vous de jouer ! Expérimentez, créez vos propres projets, et laissez la puissance de la **programmation fonctionnelle** transformer vos applications Dart et Flutter en véritables œuvres professionnelles.